

Rashtreeya Sikshana Samithi Trust
RV UNIVERSITY
School of Computer Science and Engineering
Bengaluru – 560059



**NATURAL LANGUAGE PROCESSING
TECHNIQUES AND APPLICATIONS**

COURSE CODE: CS3255

VI SEMESTER B.Sc (Hons.)

LAB INSTRUCTORS' MANUAL

2026-2027

Prepared by:

Verified by: Program Director, BSc

Vision and Mission of the School of Computer Science and Engineering

Vision

To be a pioneering school of Computer Science and Engineering committed to fostering liberal education and empowering the next generation of technologists to make a positive global socio-economic impact.

Mission

SoCSE M1: To be a pioneer in computer science education, fostering multidisciplinary research, innovation and entrepreneurship.

SoCSE M2: To provide state-of-the-art facilities and dynamic curriculum that enables exemplary pedagogy, integrating advanced technology, theoretical foundations, and hands-on applications to address real-world challenges.

SoCSE M3: To promote diverse communities of faculty and students through national and international academic, industry collaborations contributing to institution-building.

SoCSE M4: To cultivate a generation of ethical, self-motivated, and empathetic problem solvers dedicated to achieving sustainable development goals.

PROGRAM EDUCATIONAL OBJECTIVES (PEOs)

- PEO1.** Graduates will be able to apply their knowledge and critical thinking to solve real-world problems, demonstrate leadership and ethics, and contribute to societal progress through lifelong learning and interdisciplinary collaboration.
- PEO2.** Graduates will drive technological innovation and entrepreneurship, leveraging multidisciplinary research to develop sustainable solutions that address societal challenges.
- PEO3.** Graduates will engage in global academic and industry collaborations, utilizing advanced computing technologies to lead transformative initiatives for organizational development.

PROGRAM SPECIFIC OUTCOMES (PSOs)

- PSO1.** Professional Readiness for Evolving Tech Ecosystems
Demonstrate readiness for professional roles, higher studies, research, or entrepreneurship by continuously adapting to emerging technologies.
- PSO2.** Tech-Driven Community Engagement
Engage with societal challenges to develop impactful, technology-driven solutions that serve and empower communities.
- PSO3.** Building Practical Solutions with Computing Principles
Apply computing concepts to build reliable systems and solve challenges in emerging areas considering practical and ethical aspects.

PROGRAM OUTCOMES (POs)

A graduate of the BSc will demonstrate

- PO1:** Application of Knowledge: Apply knowledge from the core/interdisciplinary areas and develop skills to solve complex real-world problems.
- PO2:** Critical Thinking & Creativity: Analyze and evaluate the existing situations, practices and evidence to develop innovative multi-perspective solutions.
- PO3:** Research & Innovation: Cultivate a keen sense of observation and enquiry using appropriate methodology to develop new ideas, processes & solutions in response to contemporary challenges.
- PO4:** Digital Transformation: Instill an ability for confident, critical and responsible use of digital technologies for learning, work and participation in society.

- PO5:** Communication & Interpersonal Skills: Develop skills to effectively express thoughts and ideas both orally and in writing to people from all sections of society.
- PO6:** Leadership & Collaborative Teamwork: Exhibit skills to set goals and guide people to align effectively in building a team to achieve the goals.
- PO7:** Project Management & Entrepreneurial Mindset: Foster an enterprising spirit by working on diverse projects, demonstrating time management, resource allocation and risk assessment capabilities to create value.
- PO8:** Lifelong Learning: Engage in continuous self-directed learning to adapt to evolving technologies, societal changes and emerging global challenges.
- PO9:** Global Perspective & Cultural Awareness: Acquire knowledge about the values and beliefs of diverse cultures, effectively engaging with empathy.
- PO10:** Ethical Reasoning & Social Responsibility: Demonstrate moral and value-based principles while being socially conscious and accountable.
- PO11:** Civic Engagement & Community Service: Participate actively in civic life exhibiting a commitment to community service, social justice and the common good.
- PO12:** Environmental Sustainability: Show responsibility to manage and work towards achieving the Sustainable Development Goals (SDGs).

Course Outcome

CO1	Apply fundamental NLP techniques to process and clean raw text data using standard tools and libraries.
CO2	Implement and evaluate basic statistical and rule-based models for language modelling, part-of-speech tagging, and syntactic analysis.
CO3	Analyse the different word representation techniques including traditional and contextual embeddings.
CO4	Implement deep learning models to solve real-world NLP tasks such as summarization, translation, chatbots, and sentiment analysis while considering ethical aspects.

Course Outcomes (COs)	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO1 0	PO1 1	PO1 2
CO1: Apply fundamental NLP techniques to process and clean raw text data using standard tools and libraries.	2	2	1	2	2	1		2				
CO2: Implement and evaluate basic statistical and rule-based models for language modelling, part-of-speech tagging, and syntactic analysis.	3	2	3	2	1	2		2				
CO3: Analyse different word representation techniques, including traditional and contextual embeddings.	2	3	3	2	1	1		2				
CO4: Implement deep learning models to solve real-world NLP tasks such as summarization, translation, chatbots, and sentiment analysis, considering ethical aspects.	3	3	3	3	2	2	2	2	3	3	1	1

LIST OF PROGRAMS

Sl. No.	Program
1.	Text Processing and Normalization
2.	Pattern Matching using Regular Expressions and Edit Distance.
3.	N-Gram Language Model with Smoothing Perplexity.
4.	POS Tagging and Named Entity Recognition (NER).
5.	Parsing sentences using Constituency Parsers.
6.	Text Vectorization and Similarity Analysis.
7.	Word Embeddings with Word2Vec, GloVe and FastText.
8.	Contextual Embeddings using BERT and ELMo.
9.	Fine Tuning BERT for Text Classification.
10.	Case Study – Building a Chatbot or QA System.

Lab Assignment 1: Foundations of NLP

Text processing and Normalization

Topic: The NLP Pipeline, Morphology, and Regular Expressions

Submission Format: Colab Notebook (.ipynb)

Scenario

You have been hired as a **Junior NLP Engineer** at "VoiceTech Solutions." Your first task is to process a raw dataset of customer feedback for a new product launch. The data is messy, contains sensitive information, and needs to be structured before it can be fed into an AI model.

Input Text (The Data):

"UNBELIEVABLY... the driver, Alex, arrivED on 12/05/2024!
He was super helpful. However, the ride cost \$45.50 which is pricey.
Contact support@taxi-tech.com or call (555) 019-2834 for refunds.
I was unhappy with the wait time though."

Part A: Theoretical Analysis (5 Marks)

Answer this in a text cell or comment block.

Select the sentence: "The driver arrived on time."

Perform a Linguistic Analysis of this sentence using the Levels of NLP we discussed in class:

1. **Morphology:** Break down the word "arrived" into its root and suffix. What does the suffix tell you?
2. **Lexical Analysis:** Identify the Part of Speech (POS) for "driver" and "arrived".
3. **Syntax:** Group the words into a Noun Phrase (NP) and a Verb Phrase (VP).
4. **Semantics:** What is the literal meaning of 'ride' here?
5. **Pragmatics:** If a user writes this in a 1-star review, what is the implied "intent"? (Sarcasm vs. Fact).

Part B: Practical Implementation (15 Marks)

*Write a Python script using the `re` module to perform the following tasks on the **Input Text** provided in the Scenario.*

Task 1: The Pipeline (Cleaning)

1. **Normalize** the entire text (lowercase all letters).
2. **Clean** the text by removing special punctuation (like ... and !), but **keep** the dollar signs (\$), dots for emails/prices (.), and parenthesis for phone numbers.

Task 2: Information Extraction (Regex)

Write specific Regular Expressions to extract and print the following:

1. **The Date:** Find the date in DD/MM/YYYY format.
2. **The Price:** Extract the cost (must handle the \$ symbol and decimals).
3. **The Entities:**
 - Extract the **Email Address**.

- Extract the **Phone Number** (Note: Use the *Complex Pattern* that handles parenthesis like (555)).

Task 3: Morphological Mining

Using Regex, analyze the word structure to find:

1. **The Agent (The Doer):** Find the word ending in -er (e.g., "driver").
2. **The Action (Past Tense):** Find the word ending in -ed.
3. **The Negation:** Find the word with a negative prefix (specifically un- or dis-).

Bonus Challenge (Optional)

Modify your Phone Number regex so it can capture **BOTH** of these formats simultaneously from the text:

- (555) 019-2834
- 555-019-2834

(Hint: Remember how to make the parenthesis optional?)

Instructions & Helpful Hints

General Instructions:

- Don't forget to import re at the start of your script!
- Always test your code with the specific **Input Text** provided above.
- Print your output clearly for each task (e.g., print("Emails found:", emails)).

Hints for Part A (Theory):

- **Morphology:** Look closely at the suffix "-ed". Does it change the meaning to the past, present, or future?
- **Syntax:** A **Noun Phrase (NP)** usually contains the main subject (Who?), and a **Verb Phrase (VP)** contains the action (What did they do?).

Hints for Part B (Coding):

- **Normalization:** Python's .lower() string method is the easiest way to normalize case.
- **Cleaning:** When using re.sub(), you can use the caret ^ inside square brackets [] to mean "NOT".
 - *Example:* [^a-z] matches anything that is *not* a lowercase letter.
- **Escaping Characters:** Special symbols like \$ and . have special meanings in Regex. To find the literal character, put a backslash before it (e.g., \. matches a real dot).
- **Word Boundaries:** When searching for words ending in "-ed" or "-er", use \b to make sure you match the whole word.
 - *Example:* \w+ed\b matches "arrived" but avoids matching inside a word like "edge".
- **Negation Pattern:** To match *either* "un" or "dis", use the pipe symbol | inside a group.

Lab Assignment 2: Scenario-Based Text Preprocessing, Normalization, spaCy & Edit Distance

Pattern Matching using Regular Expressions and Edit Distance

Scenario

You are working with customer reviews collected from an Indian e-commerce platform. The reviews are informal, contain spelling mistakes, inconsistent casing, and filler words. Before further analysis, the text must be cleaned and standardized.

Input Text (Use This Exactly)

Raw Customer Review	Thiss PHONE is Amazng!!! I bought it last wek, and the battry life is sooo goood but the camra quality isnt that greaat.
---------------------	--

Tasks

1. Import required libraries: nltk, numpy, spacy, and supporting modules.
2. Download and load required NLTK resources and spaCy English model.
3. Apply text normalization on the given input text (lowercasing, punctuation removal, whitespace cleanup).
4. Tokenize the normalized text using both NLTK and spaCy.
5. Remove stop-words using both NLTK and spaCy.
6. Apply stemming using NLTK Porter Stemmer.
7. Apply lemmatization using NLTK WordNet Lemmatizer and spaCy lemmatizer.
8. Select one misspelled word and its correct form from the input text.
9. Compute edit distance using NLTK with substitution cost = 1 and = 2.
10. Print outputs clearly after each step and write a brief explanation.

Lab Assignment 3: Statistical Language Modeling: N-Grams, Smoothing, and perplexity

N-Gram Language Model with Smoothing Perplexity N

Welcome to Lab 3! □

In previous labs, we cleaned text and played with patterns. Now, we move to the core of NLP: **Language Modeling**. Your goal is to build a system that can predict the probability of the next word in a sequence.

This lab is based directly on the concept of **Markov Assumptions** and uses the techniques from your "*N_Gram_Language_Model*" notebook. We will also tackle the problem of "Zero Probability" using smoothing.

Part A: Vocabulary & N-Gram Construction

Objective: Convert raw text into probabilistic counts. (Reference: Jurafsky & Martin Ch. 3)

1. **Data Preparation:** Load the Gutenberg corpus (e.g., *Shakespeare's Macbeth* or *Austen's Emma*) using NLTK.
 - Create a function `build_vocab(corpus, min_freq=2)` that replaces words occurring less than `min_freq` times with an `<UNK>` token.
 - Append start `<s>` and end `</s>` tokens to every sentence.
2. **Generating N-Grams:** Write a function that takes a list of tokens and an integer N , and returns a dictionary of counts for all N-grams.
 - Example Output for $N = 2$: `{('the', 'cat'): 5, ('cat', 'sat'): 3, ...}`

Part B: Probability & Smoothing

Objective: Handle sparsity in training data.

3. **Unsmoothed Probability (P_{MLE}):** Implement a class `NGramModel` that calculates the Maximum Likelihood Estimation for a word w_n given history w_{n-1} :

$$P(w_n|w_{n-1}) = \frac{C(w_{n-1}, w_n)}{C(w_{n-1})}$$

Test this by calculating the probability of a common phrase (e.g., "the end") versus a non-existent one.

NLP Helper Series Page 1

RV University Natural Language Processing (CS3255) Lab Assignment 03

4. **Laplace Smoothing (Add-1):** To prevent the model from assigning zero probability to unseen events

(which crashes Perplexity), implement **Laplace Smoothing**:

$$P_{\text{Laplace}}(w_n|w_{n-1}) = \frac{C(w_{n-1}, w_n) + 1}{C(w_{n-1}) + V}$$

where V is the size of your vocabulary.

Part C: Evaluation (Perplexity)

Objective: Quantify how well your model predicts text.

- 5. Perplexity Calculation:** Implement the Perplexity ($P P$) metric. To avoid underflow (num bers becoming too small for the computer), use the **log-probability** form:

$$P P(W) = \exp \left(- \frac{1}{N} \sum_{i=1}^N \ln P(w_i|w_{i-1}) \right)$$

Task: Compare the perplexity of your Unsmoothed model vs. your Smoothed model on a held-out test set. (*Note: The unsmoothed model will likely give infinite perplexity on unseen data!*)

Part D: Creative Generation

Objective: Have fun with the model!

6. **Sentence Generator:** Use your Smoothed Bigram Model to generate 5 random sentences. Start with the token <s> and sample the next word based on the probability distribution until you hit </s> or a length limit.
7. **Analysis:** Compare the grammatical correctness of sentences generated by a **Unigram** model vs. a **Bigram** model. Why does the Bigram model usually make more sense locally?

Part E: Comparative Analysis

Objective: Analyze how different training data affects the model.

8. **Different Corpus, Different Results:** Load a second, distinct corpus using NLTK (e.g., Brown for news/editorial or Webtext for casual conversation).
 - Train a Bigram model on this new corpus.
 - Compare the **generated sentences** from the Gutenberg model vs. the new model. How does the style/tone differ?
 - **Cross-Domain Experiment:** Calculate the perplexity when you test your **Shakespeare/Gutenberg** model on the **News/Webtext** test set. Does the perplexity go up or down? Explain why domain mismatch causes this difference.

NLP Helper Series Page 2

RV University Natural Language Processing (CS3255) Lab Assignment 03

Submission Guidelines

- Submit Lab03_YourName_USN.ipynb.
- **Required Output:** Perplexity scores (same domain vs. cross-domain) and 3 sample sentences from each corpus.
- **Reflection:** In a markdown cell, explain why $N = 3$ (Trigram) increases sparsity compared to $N = 2$.

NLP Helper Series Page 3

Lab Assignment 4: POS Tagging & Named Entity Recognition

Contents

1 Introduction 1

2 Part 1: Part-of-Speech (POS) Tagging

2.1 What is POS Tagging?

2.2 How does it work? (Hidden Markov Models)

2.3 Implementation: POS Tagging with NLTK and spaCy

3 Part 2: Named Entity Recognition (NER)

3.1 What is NER?

3.2 Implementation: NER with spaCy

4 Part 3: Evaluation Metrics 4.1 Implementation: Calculating Metrics using Scikit-Learn

5 Conclusion & Student Challenge

1 Introduction

Welcome to the structural backbone of language! While we often think of language as a stream of words, to a computer, it is initially just a string of characters. To derive meaning, we must teach the machine to recognize the *role* each word plays and the *entities* it refers to. This lesson covers two fundamental tasks in the NLP pipeline:

1. **Part-of-Speech (POS) Tagging:** Identifying grammatical roles (nouns, verbs, adjectives).
2. **Named Entity Recognition (NER):** Identifying real-world objects (people, companies, locations).

These steps are crucial for downstream tasks like translation, question answering, and sentiment analysis [Jurafsky & Martin, 2007].

2 Part 1: Part-of-Speech (POS) Tagging

2.1 What is POS Tagging?

POS tagging is the process of labeling each word in a text corpus with a specific part of speech tag based on its definition and context.

Imagine a movie script. You have a list of words (the cast), but you don't know who plays what role.

- **Noun:** The Actors (People, Places, Things)
- **Verb:** The Actions (Run, Jump, Speak)
- **Adjective:** The Costumes/Makeup (Describing the actors)

POS Tagging is the "Casting Director" that assigns every word its correct role in the scene. Without this, the computer doesn't know if "book" is a physical object (noun) or an action (verb, as in "book a flight").

2.2 How does it work? (Hidden Markov Models)

While modern systems use Deep Learning (Transformers), traditional robust systems use **Hidden Markov Models (HMM)**.

An HMM relies on two probabilities to guess a tag:

1. **Emission Probability:** How likely is a word to be a specific tag? (e.g., "The" is almost always a DET (Determiner)).
2. **Transition Probability:** How likely is one tag to follow another? (e.g., A DET is very likely to be followed by a NOUN, and rarely by a VERB).

The **Viterbi Algorithm** is then used to find the most probable sequence of tags for a sentence by calculating the path with the highest probability [Manning & Schütze, 1999].

2.3 Implementation: POS Tagging with NLTK and spaCy

Below is a complete, runnable Python script comparing two popular libraries.

```

1 import nltk
2 import spacy
3
4 # --- SETUP: Download necessary NLTK data ---
5 # (Uncomment these lines if running for the first time)
6 # nltk.download('punkt ')
7 # nltk.download('averaged_perceptron_tagger ')
8 # nltk.download('universal_tagset ')
9
10 # Sample Text
11 text = "Apple is looking at buying U.K. startup for $1 billion." 12
13 # =====
14 # APPROACH 1: Using NLTK (Rule -based/Stochastic)
15 # =====
16
17
18 # Step 1: Tokenization
19 tokens = nltk.word_tokenize(text)
20
21 # Step 2: Tagging
22 # NLTK uses the Penn Treebank tagset by default (NN, VB, JJ, etc.) 23 nltk_tags =
nltk.pos_tag(tokens)
24
25 print(f"{'Word':<15} {'Tag':<10} {'Description':<10}")
26 print("-" * 40)
27
28 for word, tag in nltk_tags:
29     # We can simplify tags (optional) or print as is
30     print(f"{'word':<15} {'tag':<10}")
31

```

```

32
33 # =====
34 # APPROACH 2: Using spaCy (Deep Learning based)
35 # =====
36 print("\n--- spaCy POS Tagging ---")
37
38 # Load the small English model
39 # Run in terminal: python -m spacy download en_core_web_sm
40 nlp = spacy.load("en_core_web_sm")
41
42 # Process the text
43 doc = nlp(text)
44
45 print(f"{'Word':<15} {'POS':<10} {'Tag':<10} {'Explanation':<10}") 46 print("-" * 60)
47
48 for token in doc:
49     # token.pos_ is the coarse -grained tag (Universal POS)
50     # token.tag_ is the fine -grained tag (Penn Treebank)
51     print(f"{'token.text':<15} {'token.pos_':<10} {'token.tag_':<10} {'spacy.explain(token.tag_)}")

```

Listing 1: Complete POS Tagging Workflow

3 Part 2: Named Entity Recognition (NER)

3.1 What is NER?

Named Entity Recognition involves locating and classifying named entities in unstructured text into predefined categories such as **PERSON**, **ORG** (Organization), **GPE** (Geopolitical Entity), **DATE**, etc.

Imagine reading a newspaper and using a yellow highlighter for names of people, a blue one for companies, and a green one for locations. NER automates this highlighting process. It turns "The CEO of Google visited India" into:

- "Google" → **ORG**
- "India" → **GPE**

3.2 Implementation: NER with spaCy

spaCy is the industry standard for NER due to its speed and accuracy.

```

import spacy
from spacy import displacy

# Load model
nlp = spacy.load("en_core_web_sm")

# Sample text with rich entities
text = "Elon Musk founded SpaceX in 2002. It is based in California."

# Process text
doc = nlp(text)

print(f"\n--- Named Entity Recognition ---")
print(f'Entity':<20} {'Label':<10} {'Explanation':<10})
print("-" * 60)

for ent in doc.ents:
    print(f'ent.text: <20} {ent.label_}: <10} {spacy.explain(ent.label_)}')

# --- VISUALIZATION CODE ---
# If running in Jupyter Notebook , use:
# displacy.render(doc, style="ent", jupyter=True)

# If running as a script , this generates an HTML file:
html = displacy.render(doc, style="ent", page=True)
with open("ner_visualization.html", "w", encoding="utf-8") as f: f.write(html)

print("\nVisualization saved to 'ner_visualization.html'")

```

Listing 2: NER Extraction and Visualization

4 Part 3: Evaluation Metrics

How do we know our tagger is good? We cannot just use simple accuracy (correct tags / total tags) because some tags (like "O" or "Noun") are far more common than others.

1. **Precision:** Of all the words we labeled as PERSON, how many were actually people? 2.

Recall: Of all the people in the text, how many did we manage to find?

3. **F1-Score:** The harmonic mean of Precision and Recall. This is the standard metric for NER systems.

4.1 Implementation: Calculating Metrics using Scikit-Learn

In practice, we use the 'scikit-learn' library to calculate these metrics. In this example, we compare a list of "Ground Truth" tags (what a human expert would say) against a list of "Predicted" tags (what our model guessed).


```

from sklearn.metrics import classification_report , accuracy_score

# Sample Data:
# 'y_true' represents the actual correct tags for a sentence
# 'y_pred' represents what our model predicted
# Example Sentence: "Google is a big company"
# Correct Tags: [ORG, VERB , DET, ADJ, NOUN]

y_true = ["ORG", "VERB", "DET", "ADJ", "NOUN", "NOUN", "PUNCT"] y_pred = ["ORG",
"VERB", "DET", "NOUN", "NOUN", "NOUN", "PUNCT"] # Note the error: The model
predicted "big" (ADJ) as a "NOUN"

print("--- Evaluation Metrics ---\n")

# 1. Accuracy
accuracy = accuracy_score(y_true , y_pred)
print(f"Accuracy: {accuracy:.2f}")

# 2. Classification Report (Precision , Recall , F1)
# This generates a table showing metrics for every single tag type
report = classification_report
(y_true , y_pred , zero_division=0) print("\nClassification Report:\n")
print(report)

print("\nAnalysis:")
print("Notice how the ADJ tag has 0.00 precision/recall because our model missed it entirely.")
print("The NOUN tag has high recall but lower precision because we over - predicted it.")

```

Listing 3: Calculating Precision Recall and F1

5 Conclusion & Student Challenge

We have moved from treating words as simple strings to understanding their grammatical roles and real-world identities.

Challenge for Students: Modify the code above to process a news article (copy-paste a paragraph). Count which organization (**ORG**) appears most frequently in the text. This is a basic form of "Entity Linking" or "Trend Analysis."

References

- Jurafsky, D., & Martin, J. H. (2007). *Speech and Language Processing*. Prentice Hall.
- Manning, C. D., & Schütze, H. (1999). *Foundations of Statistical Natural Language Processing*. MIT Press.
- spaCy Documentation. (2025). <https://spacy.io/usage/linguistic-features> 5

NLP Helper - RVU POS Tagging & NER - RV University NLP Syllabus (Batch 2023).

Lab Assignment 4: Applied POS Tagging & NER

Assignment Overview

In this lab assignment, you will apply the concepts of Part-of-Speech tagging and Named Entity Recognition to solve three distinct industry-relevant problems. You are expected to use Python, 'NLTK', 'spaCy', and 'scikit-learn'.

1 Task 1: The "Sentiment Feature" Extractor (POS Tagging)

Context: You have been hired by an e-commerce giant, "ShopSmart." They have millions of reviews but don't just want to know if a review is positive or negative; they want to know *what* features people are talking about and *how* they describe them.

The Linguistic Logic:

- **Nouns** usually represent product features (e.g., "battery", "screen").
- **Adjectives** represent the sentiment/opinion (e.g., "terrible", "bright").

Problem Statement: Write a Python script using spaCy that takes the following customer review and extracts all pairs of (**Adjective, Noun**) that appear in the text. **Input Text:**

"The new phone has a fantastic camera but the battery life is terrible. The screen is bright , but the price is high ."

Expected Output Format:

Feature Pairs Detected:

- (fantastic, camera)
- (terrible, life)
- (bright, screen)
- (high, price)

Hint: Iterate through tokens. If a token is a NOUN, check if its children (or ancestors) are ADJs, or simply iterate and print all ADJ+NOUN sequences.

Task 2: The GDPR Redactor (NER)

Context: A legal firm, "SafeLaw," is automating the processing of court documents. To comply with GDPR and privacy laws, they must automatically redact (censor) names of people and specific locations before the documents are made public.

Problem Statement: Create a function `redact_sensitive_info(text)` using `spaCy`. The function should detect all entities labeled **PERSON** or **GPE** (Geopolitical Entity) and replace them with the string "[REDACTED]".

Input Text:

"Startups in Bangalore are growing fast. Mr. Rajesh Kumar met with Sarah Jones in London to discuss the merger ."

Expected Output:

"Startups in [REDACTED] are growing fast. Mr. [REDACTED] met with [REDACTED] in [REDACTED] to discuss the merger ."

3 Task 3: Model Auditing (Metrics)

Context: A hospital deployed a chatbot to identify medications in patient notes. However, doctors are complaining that it misses too many drug names. You need to audit the model's performance.

Problem Statement: Given the following lists of "Ground Truth" (what the doctors said) and "Predictions" (what the bot found), calculate the **Precision**, **Recall**, and **F1-Score** for the **"DRUG"** class.

Data:

```
# True tags for a specific sentence
y_true = ["O", "DRUG", "O", "DRUG", "O", "O"]

# Model predictions
y_pred = ["O", "O", "O", "DRUG", "O", "DRUG"]
```

Questions to Answer in your Report:

1. Calculate Precision, Recall, and F1 using 'sklearn.metrics'.
2. Based on the results, is the model suffering from *High False Positives* or *High False Negatives*? Explain what this means for the hospital (e.g., is it dangerous?).

Submission Instructions

1. Complete the assignment in a Google Colab notebook.
2. **Do not submit the .ipynb file.** Instead, submit the **shared link** to your Colab notebook.
3. Ensure the sharing settings are set to **"Anyone with the link can view"** (View Mode).
4. Include a markdown cell at the end with your analysis for Task 3.
5. **Notebook Naming:** YourName_SRN_Lab2_NLP

Lab Assignment 5: Parsing Sentences

using Constituency Parsers

1. Introduction: What is Constituency Parsing?

In Natural Language Processing, parsing is the process of analyzing the grammatical structure of a sentence. **Constituency parsing** (also known as phrase structure parsing) breaks down a sentence into its constituent parts based on a hierarchical structure.

It groups words into functional units or "phrases," such as Noun Phrases (NP), Verb Phrases (VP), and Prepositional Phrases (PP). The output is a parse tree where the root is the entire sentence (S), the internal nodes are the phrase labels, and the leaves are the actual words [Jurafsky & Martin, Speech and Language Processing].

2. Why, When, and How?

Why do we use it? Constituency parsing is crucial for resolving syntactic ambiguity. For example, in the sentence "I saw the man with the telescope," constituency parsing helps the machine understand whether "with the telescope" modifies the man (the man holds the telescope) or the verb "saw" (the speaker used the telescope). It explicitly maps out these phrase boundaries.

When do we use it?

- **Information Extraction:** When you need to extract complete phrases (like named entities or specific subjects) rather than just word-to-word relationships.
- **Machine Translation:** To ensure translated sentences maintain the correct grammatical phrase structures of the target language.
- **Grammar Checking:** To verify if a sentence conforms to formal grammatical rules.

How does it work? Constituency parsers use a set of grammatical rules called a **Context Free Grammar (CFG)**. A CFG defines how symbols can be grouped together. For example, a rule like $S \rightarrow NP VP$ tells the parser that a Sentence consists of a Noun Phrase followed by a Verb Phrase. Algorithms then use these rules to build trees either from the top-down (starting with S) or bottom-up (starting with the words).

Parameters in Parsing:

- **Grammar / Lexicon:** The set of production rules (e.g., $NP \rightarrow Det N$) and the vocabulary defining word categories (e.g., $N \rightarrow^0 cat^0$).
- **Parsing Algorithm:** The search strategy used to apply the grammar (e.g., recursive descent, shift-reduce).
- **Probabilities (in PCFGs):** Modern parsers use Probabilistic CFGs, where rules have associated weights/probabilities to choose the most likely parse tree when multiple valid trees exist.

3. Types of Parsers in Modern NLP (Including RVU Syllabus)

Modern NLP employs various parsing strategies, broadly categorized into constituency and dependency parsing. Based on your syllabus and current industry standards, here are the key types:

- Constituency Parsers:

- **Recursive Descent Parser:** A top-down approach that starts at the root (S) and recursively expands nodes.
- **Shift-Reduce Parser:** A bottom-up approach using a stack buffer to shift words and reduce them into phrases.
- **CYK (Cocke-Younger-Kasami) Algorithm:** A bottom-up dynamic programming parser. (Detailed in Section 4).
- **Earley Algorithm:** A top-down dynamic programming algorithm that uses a chart to keep track of partial predictions, successfully handling all CFGs.

- **Dependency Parsers:** Instead of grouping words into phrases, these draw directed links directly between words.

- Transition-based & Graph-based Parsing.

- **Deep Learning / LLM Parsers:** Modern parsers utilize Transformer architectures (like BERT) to achieve incredibly high accuracy [Kitaev and Klein, 2018].

4. Deep Dive: The CYK (Cocke-Younger-Kasami) Algorithm

The CYK algorithm is a highly efficient, bottom-up parsing algorithm that uses **dynamic programming**. Think of dynamic programming like solving a large puzzle by first assembling the edge pieces, then small clusters, and finally joining them together. CYK stores partial solutions in a table (or "chart") so it never has to compute the same phrase twice.

The Golden Rule of CYK: To use CYK, your grammar *must* be in **Chomsky Normal Form (CNF)**. This means every rule must look like one of two things:

- A Non-terminal producing two Non-terminals (e.g., $A \rightarrow B C$)
- A Non-terminal producing exactly one word (e.g., $A \rightarrow^0 \text{apple}^0$)

Complete CYK Python Implementation with Visualization: Below is the complete, runnable Python code. Copy and paste this into your environment. When you run it, the `print_chart` helper function will output the dynamic programming grid at each step so you can watch the algorithm build the sentence structure!

```
# A complete, runnable CYK Algorithm implementation in Python
# Note: The grammar is strictly in Chomsky Normal Form (CNF)
grammar = {
    'S': [['NP', 'VP']],
    'NP': [['Det', 'N']],
    'VP': [['V', 'NP']],
    'Det': [['the'], ['a']],
    'N': [['cat'], ['dog']],
    'V': [['chased'], ['saw']]
}
```

```

def cyk_parse(words, grammar):
    n = len(words)
    # Initialize an empty N x N chart (a 2D list of sets)
    chart = [[set() for _ in range(n)] for _ in range(n)]

    # Helper function to print the chart visually in the console def
    print_chart(step_name):
        print(f"\n--- {step_name} ---")
        header = f"{'':<10}" + "".join([f"{'w':<10}" for w in words])
        print(header)
        print("-" * len(header))
        for row in range(n):
            row_str = f"{'words[row]:<10}"
            for col in range(n):
                cell = ", ".join(chart[row][col]) if chart[row][col] else "."
                row_str += f"{'cell:<10}"
            print(row_str)

    # Step 1: Fill the diagonal (Lexical rules: A -> word)
    for i, word in enumerate(words):
        for lhs, rhs_list in grammar.items():
            for rhs in rhs_list:
                if len(rhs) == 1 and rhs[0] == word:
                    chart[i][i].add(lhs)

    print_chart("Step 1: After Filling Lexical Diagonal")

    # Step 2: Fill the rest of the chart (Syntactic rules: A -> B C) for length in
    range(2, n + 1): # Span length of the phrase
        for i in range(n - length + 1): # Start index of the span
            j = i + length - 1 # End index of the span
            for k in range(i, j): # Split point to divide the span in two

                # Check combinations from left part (i to k)
                # and right part (k+1 to j)
                for lhs, rhs_list in grammar.items():
                    for rhs in rhs_list:
                        if len(rhs) == 2:
                            B, C = rhs
                            # If B is in left span and C is in right span
                            if B in chart[i][k] and C in chart[k+1][j]:
                                chart[i][j].add(lhs)

    print_chart(f"Step {length}: After Span Length {length}")

    # Step 3: Check if the start symbol 'S' spans the entire sentence # The top-right cell of
    the chart represents the whole sentence (0 to n-1) if 'S' in chart[0][n-1]:
        print(f"\nSuccess! '{' '.join(words)}' is grammatically valid.")
        return True

    else:
        print(f"\nFailed to parse: '{' '.join(words)}'")

```

```

        return False

# Let's run it!
sentence = "the cat chased a dog".split()
cyk_parse(sentence, grammar)

```

5. Visualizing the Parsing Process

Understanding parsing algorithms is much easier when we can visualize the structures they create. Here are two ways to visualize what we just computed:

1. The CYK Dynamic Programming Chart

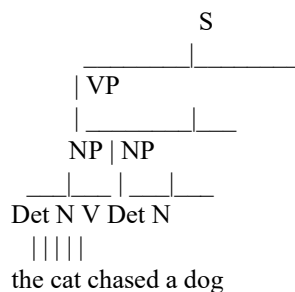
If we trace the Python code from Section 4 on paper, we build an $N \times N$ matrix. The diagonal represents individual words (Length 1 spans), and we combine cells to move towards the top right corner to find the start symbol **S** (Length 5 span).

Start \ End	0 (the)	1 (cat)	2 (chased)	3 (a)	4 (dog)
0 (the)	Det	NP	-	-	S
1 (cat)		N	-	-	-
2 (chased)			V	-	VP
3 (a)				Det	NP
4 (dog)					N

Table 1: CYK Chart for “the cat chased a dog”

2. The Parse Tree

Once the chart is successfully completed, the algorithm extracts the syntactic tree. In NLTK, calling `tree.pretty_print()` generates a beautiful ASCII tree in your console, while `tree.draw()` opens an interactive graphical window. Here is the structural visualization for our sentence:



6. Parsing with NLTK and CNF Conversion

While the above shows the raw logic of CYK, practical NLP usually relies on libraries. NLTK pro

vides built-in tools to handle grammars and can even convert a standard CFG into Chomsky Normal Form automatically!

```
import nltk
from nltk import CFG
from nltk.parse import ChartParser

# A natural grammar (NOT in strict CNF due to unit productions like VP -> V)
natural_grammar = CFG.fromstring("""
    S -> NP VP
    NP -> Det N | PropN
    VP -> V NP | V
    Det -> 'the' | 'a'
    N -> 'student' | 'book'
    PropN -> 'John'
    V -> 'read' | 'studies'
""")

# NLTK can automatically convert this to CNF!
# This removes unit productions (like VP -> V) and binarizes the rules.
cnf_grammar = natural_grammar.chomsky_normal_form()

print("Original Grammar Rules:", len(natural_grammar.productions()))
print("CNF Grammar Rules:", len(cnf_grammar.productions()))

# Now we parse using the strictly CNF grammar
parser = ChartParser(cnf_grammar)
tokens = "John read a book".split()

for tree in parser.parse(tokens):
    # The output tree will show the intermediate dummy nodes
    # created during the CNF conversion!
    tree.pretty_print()
```

7. Critical Thinking Assignment

The Ambiguity Challenge

Natural language is inherently ambiguous. Consider the famous sentence: *"I shot an elephant in my pajamas."*

Your Task:

1. **Chomsky Normal Form Analysis:** Why is it difficult to convert a grammar that naturally handles ambiguous prepositional phrase attachments (like "in my pajamas") into strict Chomsky Normal Form for the CYK algorithm?
2. **Code Modification:** Modify the CFG in the NLTK code provided in Section 6 so that it can successfully parse the sentence: ["i", "shot", "an", "elephant", "in", "my", "pajamas"]. Ensure your grammar rules make the ChartParser output **two distinct parse trees**.
3. **Semantic Explanation:** Write a short paragraph explaining the structural difference between the two generated trees. What grammatical attachment changes, and how does that change the real-world meaning of the sentence?

References:

1. Jurafsky, D., & Martin, J. H. (2008). *Speech and Language Processing*. 2. *NLTK Documentation* (nltk.org/api/nltk.parse.html).

Lab Assignment 6: Text Vectorization and Similarity Analysis

Introduction

Welcome to Lab 6! Today, we tackle one of the most fundamental questions in Natural Language Processing: *How do we make computers understand words?*

Imagine trying to explain the taste of an apple to a computer. You can't. But you *can* assign numerical coordinates to its properties (Sweetness: 0.8, Crunchiness: 0.9, Acidity: 0.4). **Text Vectorization** is exactly this process—converting text strings into numerical arrays (vectors) in a multi-dimensional space. Once texts are turned into vectors, we can measure how close they are to each other using **Similarity Analysis**.

1 Method 1: Bag of Words (BoW)

The Concept: The Bag of Words model is the simplest form of text vectorization. Think of it like looking at the ingredients of a recipe while completely ignoring the cooking instructions (the order of the words). It treats every document merely as a "bag" of its words. It works by creating a vocabulary of all unique words in your dataset and then counting how many times each word appears in each document. Because it extracts each word as a feature without regard to context, it can sometimes struggle with polysemous words (words with multiple meanings) [IBM, 2024], but it remains an incredibly robust baseline.

When to use it: BoW is great for simple classification tasks (like basic spam filtering) where the mere presence of certain words is a strong indicator of the class.

Which Library: scikit-learn provides CountVectorizer, which efficiently handles the creation of sparse matrices (matrices that are mostly zeros, saving memory).

Step-by-Step Code

```
1 from sklearn.feature_extraction.text import CountVectorizer
2 import pandas as pd
3
4 # 1. Define our corpus
5 corpus = [
6     "I love machine learning.",
7     "Machine learning is amazing.",
8     "Deep learning and machine learning are related."
9 ]
10
11 # 2. Initialize the Vectorizer and transform the text
12 vectorizer = CountVectorizer()
13 bow_matrix = vectorizer.fit_transform(corpus)
14
15 # 3. View the results as a DataFrame for readability
16 vocabulary = vectorizer.get_feature_names_out()
17 df_bow = pd.DataFrame(bow_matrix.toarray(), columns=vocabulary)
18
19 print("Bag of Words Matrix:\n", df_bow)
```

The Shortcoming: If you look closely at the matrix, BoW treats every word equally. A highly common word like "is" or "and" might get a high count, overshadowing important words like "machine".

2 Method 2: TF-IDF (Term Frequency-Inverse Document Frequency)

The Concept: TF-IDF solves the BoW problem. It consists of two parts:

- **Term Frequency (TF):** How often does a word appear in this specific document? This asks, "Is this document heavily about this specific word?" (Higher is better).
- **Inverse Document Frequency (IDF):** How rare is this word across *all* documents? This asks, "Does this word help me distinguish this document from the rest?" (Rarer is better).

By multiplying $TF \times IDF$, words that are common in one document but rare globally (like "ma chine") get a high score. Conversely, words common everywhere (like "the" or "is") are penalized to near zero [Manning & Schütze, 1999].

Real-World Analogy: Imagine you search a database for "how to configure SSL". The words "how" and "to" will appear in almost every document, so their IDF score drops to zero. The word "SSL", however, is rare globally but frequent in specific IT documents, so it gets a massive TF-IDF boost, helping the search engine find the exact technical guide you need [Towards AI, 2026].

Step-by-Step Code

```
1 from sklearn.feature_extraction.text import TfidfVectorizer
2
3 # 1. Initialize TF-IDF Vectorizer
4 tfidf_vectorizer = TfidfVectorizer()
5
6 # 2. Fit and transform the same corpus
7 tfidf_matrix = tfidf_vectorizer.fit_transform(corpus)
8
9 # 3. View the results
10 df_tfidf = pd.DataFrame(tfidf_matrix.toarray(), columns=vocabulary)
11 print("TF-IDF Matrix:\n", df_tfidf.round(2))
```

3 Similarity Analysis and Visualization

The Concept: Now that our texts are vectors, we can calculate how similar they are. We use **Cosine Similarity**, which measures the angle between two vectors in a multi-dimensional space.

Geometric Intuition: Think of vectors as arrows pointing out from an origin point.

- If two texts are highly similar, their arrows point in the exact same direction (an angle of 0 degrees). The cosine of 0 is **1.0**.
- If they share absolutely no words, their arrows are at a right angle (90 degrees). The cosine of 90 is **0.0**.

Crucially, Cosine Similarity focuses on the *angle*, not the *length* (magnitude) of the vectors. This means a short 10-word summary can be found highly similar to a 1,000-word essay as long as their word distributions align [PingCAP, 2024].

Visualization: In the industry, we rarely look at raw matrices. We use **Heatmaps** to visualize similarity. Libraries like seaborn apply a color gradient to matrices, allowing us to instantly spot clusters of similar documents.

Step-by-Step Code

```

1 from sklearn.metrics.pairwise import cosine_similarity
2 import seaborn as sns
3 import matplotlib.pyplot as plt
4
5 # 1. Calculate the similarity between all documents
6 # This creates a 3x3 matrix where cell [i, j] is the similarity of doc i and doc j
7 similarity_matrix = cosine_similarity ( tfidf_matrix )
8
9 # 2. Visualize using Seaborn
10 plt.figure(figsize=(6, 4))
11 # annot=True shows the numbers , cmap='YlGnBu' applies a Yellow -Green -Blue color scale
12 sns.heatmap( similarity_matrix , annot=True , cmap='YlGnBu',
13 xticklabels=["Doc 0", "Doc 1", "Doc 2"],
14 yticklabels=["Doc 0", "Doc 1", "Doc 2"])
15
16 plt.title("Document Similarity Heatmap")
17 plt.xlabel("Documents")
18 plt.ylabel("Documents")
19 plt.show ()

```

4 Guided Example: Simple Document Retrieval

Before tackling the assignment, let's look at how these tools are combined to solve a real-world problem: building a rudimentary search engine.

Imagine we have a small database of articles, and a user types in a search query. We want to find the article that best matches their query using TF-IDF and Cosine Similarity.

```

1 # 1. Our Database and the User's Query
2 database = [
3 "How to bake a chocolate cake.",
4 "A guide to changing your car's oil.",
5 "The best chocolate chip cookie recipe."
6 ]
7 user_query = "chocolate recipes"
8
9 # 2. Combine them so the Vectorizer learns the full vocabulary
10 all_texts = database + [ user_query ]
11
12 # 3. Vectorize using TF-IDF
13 vectorizer = TfidfVectorizer ()
14 tfidf_matrix = vectorizer . fit_transform ( all_texts )
15
16 # 4. Separate the query vector from the database vectors
17 # tfidf_matrix[-1:] gets the very last row (our query)
18 # tfidf_matrix[: -1] gets everything EXCEPT the last row (our database)
19 database_vectors = tfidf_matrix [: -1]
20 query_vector = tfidf_matrix [ -1:]
21
22 # 5. Calculate similarity between the query and the database
23 similarities = cosine_similarity (query_vector , database_vectors )
24
25 # 6. Find the best match using argmax (which returns the index of the highest value)
26 best_match_idx = similarities.argmax ()
27 print(f"User searched for: '{user_query}'")
28 print(f"Best matching document: '{database[best_match_idx]}'")
29 print(f"Similarity Score: {similarities[0, best_match_idx]:.4f}")

```

5 Real-World Assignment: Support Ticket Triage

The Scenario: You work as a Data Scientist for an IT firm. Hundreds of support tickets come in daily. To save time, you want to automatically cluster incoming tickets with past resolved tickets so agents can see how similar problems were solved previously.

Embracing AI in Coding: In the modern tech industry, developers frequently use AI tools (like ChatGPT or GitHub Copilot) to assist with coding. For this assignment, **you are allowed (and encouraged) to use AI to help you write the code.** However, your grade depends on your ability to critically evaluate the AI's output and understand the underlying mathematics.

Your Task:

1. **The Dataset:** Use the following provided list of IT support tickets. Notice how it contains two distinct clusters (passwords and printers) and one random outlier.

```
1 support_tickets = [
2     "I forgot my password and cannot log into my account.",
3     "Need help resetting my portal password , I keep getting an error.", 4 "The printer on the 3rd
4     floor is jammed and making a weird noise.", 5 "Paper jam in the main office printer , needs
5     maintenance.", 6 "The cafeteria ran out of coffee again , please restock."
6 ]
```

2. **The Stress Test:** Add a 6th ticket to your Python list that intentionally tries to confuse the TF-IDF algorithm by mixing keywords from different clusters (e.g., "I forgot the password for the cafeteria printer portal").
3. **Implementation:** Write a Python script from scratch (you may use AI to generate it) that does the following:
 - Vectorizes your 6 tickets using both CountVectorizer (BoW) and TfidfVectorizer from scikit-learn.
 - Calculates the Cosine Similarity matrix for *both* sets of vectors.
 - Uses matplotlib and seaborn to plot the two similarity matrices side-by-side as heatmaps.
4. **AI Transparency & Reflection:** Along with your code and heatmap images, submit a short write-up answering the following questions:
 - **AI Usage:** Did you use AI to help write this code? If yes, what exact prompt(s) did you use?
 - **Code Critique:** Did the AI generate the code exactly as needed on the first try, or did you have to modify it? (e.g., did it use the correct libraries, or did you have to fix any hallucinations?)
 - **Mathematical Reflection:** Look at the heatmap row for your 6th "Stress Test" ticket. Did TF-IDF cluster it correctly with the problem it actually represents, or was it mathematically confused by the overlapping keywords? Explain *why* based on how the TF-IDF formula prioritizes exact word matches over actual sentence meaning.

References

- IBM. (2024). *What is bag of words?*. IBM Think Topics.
- Jurafsky, D., & Martin, J. H. (2007). *Speech and Language Processing*. Prentice Hall.
- Manning, C. D., & Schütze, H. (1999). *Foundations of Statistical Natural Language Processing*. MIT Press.

- PingCAP. (2024). *Understanding the Cosine Similarity Formula*.
- Towards AI. (2026). *Word Embeddings in NLP: From Bag-of-Words to Transformers*.
- Waskom, M. (2021). *seaborn: statistical data visualization*. Journal of Open Source Software, 6(60), 3021.

Why Did K-Means Cluster "AI" with "Dogs"?

When you plotted this, you likely saw the sentence "The future of technology involves AI and automation." (Document 9) grouped in the same color as the sentences about kittens, puppies, and dogs.

To human eyes, this makes no sense. But to understand why the computer did this, we have to look at the world exactly as the algorithm does. Here is the step-by-step breakdown of why this illusion occurred.

1. The TF-IDF "Blind Spot" (Vocabulary Mismatch)

TF-IDF is a brilliant algorithm, but it is strictly a lexical matching tool. It only looks for exact word overlap. It does not understand synonyms or concepts.

Let's look at Document 9:

"The future of technology involves AI and automation."

Now, look at the other tech-related sentences. They use words like "Machine learning," "Artificial intelligence," "computers," and "Data science." Even though "AI" and "Artificial intelligence" mean the exact same thing to us, TF-IDF sees them as completely unrelated mathematical features. Because Document 9 shares virtually zero significant keywords with the other tech documents, TF-IDF places it very far away from the "Tech" cluster in the high dimensional vector space [Jurafsky & Martin, 2007].

2. The PCA "Crush" (Information Loss)

Your corpus has around 80 unique words. This means `TfidfVectorizer` created an 80- dimensional space.

In your code, you used `PCA(n_components=2)` to violently crush those 80 dimensions down into just 2 dimensions so you could plot it on a flat screen.

The Analogy: Imagine shining a flashlight on a 3-dimensional house and tracing its 2D shadow on the ground. Two objects that are very far apart in the 3D house (like a bird on the roof and a dog on the porch) might look like they are touching in the 2D shadow.

By forcing 80 dimensions into 2, PCA causes massive information loss. Mathematical "orphans" like Document 9 (which don't share words with any cluster) get projected randomly into the 2D space, often landing near unrelated data points.

3. K-Means "Forced Adoption"

K-Means is a partition-based algorithm. If you tell it $k=3$, it must draw three circles that contain every

single point. It does not allow for "outliers" or "unclassified" points.

Because Document 9 was stripped of its meaning by TF-IDF and had its coordinates scrambled by PCA, it ended up sitting somewhere in the 2D graph. K-Means simply looked at its coordinates and said, "Well, you are slightly closer to the geometric center of the Dog cluster than the Machine Learning cluster, so I am putting you with the dogs."

How to Fix This for Industry Applications

If you were building this for a real project, here is how you would prevent this error:

1. Change the Order of Operations: Never cluster after PCA if you can avoid it. You should run `KMeans` directly on the `tfidf_matrix_large_cluster` (the full 80-dimensional data). Then, use PCA only for the `sns.scatterplot` visualization. This ensures the clustering algorithm has all the information before making decisions.
2. Use Contextual Embeddings: As your syllabus points out in Module 3 (Transformers), TF IDF is an older method. If you swapped `TfidfVectorizer` for a Sentence Transformer (like `all-MiniLM-L6-v2`), the neural network would inherently know that "AI" and "Artificial Intelligence" are the same concept, and it would cluster Document 9 flawlessly with the other tech sentences!

References:

Jurafsky, D., & Martin, J. H. (2007). *Speech and Language Processing*. Prentice Hall. (Chapter 20: *Statistical Models*)

Lab Assignment 7: Demystifying Word Embeddings A Journey from Word2Vec to FastText

Welcome to the World of Vector Semantics!

Hello future NLP experts! Before computers can understand language, we must translate words into a language computers natively speak: numbers. Historically, we used simple counts (like Bag of Words) or sparse vectors (One-Hot Encoding). However, these methods failed to capture *meaning*.

In this guide, we will explore **Dense Word Embeddings**—techniques that represent words as dense, continuous vectors of real numbers. By the end of this document, you will understand how Word2Vec, GloVe, and FastText work, how to code them, and you will have an assignment to test your mettle!

1 Word2Vec: The Pioneer of Context

Introduced by Tomas Mikolov and his team at Google in 2013, Word2Vec revolutionized NLP. It is based on a famous linguistic theory by John Rupert Firth (1957): *"You shall know a word by the company it keeps."*

Word2Vec comes in two architectural flavors:

- **Continuous Bag of Words (CBOW):** Predicts a target word based on its surrounding context words. (Analogy: Fill in the blank: "The cat sat on the ____").
- **Skip-gram:** Predicts the surrounding context words given a single target word. (Analogy: Given the word "mat", guess what words usually appear around it).

Step-by-Step Code: Training Word2Vec

We will use `gensim`, a robust open-source library in Python for topic modeling and document indexing.

```
1 import urllib.request
2 import re
3 from gensim.models import Word2Vec
4
5 # 1. Prepare a toy dataset (normally you'd use a large corpus like Wikipedia) 6 corpus = [
7 "the quick brown fox jumps over the lazy dog",
8 "the dog barked at the mailman",
9 "the cat sat on the mat",
10 "dogs and cats are popular pets"
11 ]
12
13 # 2. Tokenization: Clean and split sentences into words
14 def tokenize(text):
15     text = text.lower()
16     text = re.sub(r'[^\w\s]', '', text)
17     return text.split()
18
19 tokenized_corpus = [tokenize(sentence) for sentence in corpus] 20
21 # 3. Train the Word2Vec model
22 # vector_size = dimensionality of the word vectors
```



```

23 # window = maximum distance between current and predicted word 24 # min_count =
ignores all words with total frequency lower than this 25 # sg = 1 for Skip-gram, 0 for CBOW
26 print("Training Word2Vec model...")
27 w2v_model = Word2Vec(sentences=tokenized_corpus, vector_size=10, window=2, min_count
=1, sg=1)
28
29 # 4. Explore the embeddings
30 word = "dog"
31 print(f"\nVector representation of '{word}':\n", w2v_model.wv[word]) 32
33 # Find most similar words
34 print(f"\nWords most similar to '{word}':")
35 similar_words = w2v_model.wv.most_similar(word, topn=2)
36 for w, score in similar_words:
37 print(f"- {w} (score: {score:.4f})")

```

Listing 1: Training a Word2Vec model from scratch

2 GloVe: Global Vectors for Word Representation

While Word2Vec looks at local context windows (a few words to the left and right), **GloVe** (Global Vectors), developed by Stanford researchers in 2014, takes a different approach. It builds a massive *co-occurrence matrix* for the entire corpus—counting how often every word appears with every other word. It then uses matrix factorization to shrink this giant matrix down to dense vectors.

Analogy: If Word2Vec learns by reading a book sentence by sentence, GloVe learns by first making a giant spreadsheet of every time two words appear together in the whole book, and then summarizing that spreadsheet.

Step-by-Step Code: Using Pre-trained GloVe Embeddings

Training GloVe from scratch requires a C++ compiler and large memory. In practice, NLP engineers usually load *pre-trained* GloVe models.

```

1 import gensim, downloader as api
2
3 # 1. Load a pre-trained GloVe model
4 # We use a lightweight version trained on Wikipedia with 50-dimensional vectors. 5 # Note: The first time you
run this, it will download ~66MB of data. 6 print("Loading pre-trained GloVe model (this may take a
moment)...") 7 glove_model = api.load("glove-wiki-gigaword-50")
8
9 # 2. Perform Vector Arithmetic (The famous King - Man + Woman = Queen) 10 # We want to find:
vector('king') - vector('man') + vector('woman') 11 print("\nSolving the analogy: King is to Man as [?] is to
Woman") 12 result = glove_model.most_similar(positive=['king', 'woman'], negative=['man'], topn=1)
13 print(f"Answer: {result[0][0]} (Confidence: {result[0][1]:.4f})" 14
15 # 3. Check similarity between specific words
16 sim = glove_model.similarity('apple', 'banana')
17 print(f"\nSimilarity between 'apple' and 'banana': {sim:.4f}") Listing 2: Loading and using

```

pre-trained GloVe embeddings via Gensim

3 FastText: The Power of Subwords

Developed by Facebook AI in 2017, **FastText** improves upon Word2Vec by treating each word as a collection of character n-grams. For the word "apple" (with n=3), FastText breaks it down into: <ap,

app, ppl, ple, le>. The final vector for "apple" is the sum of the vectors of these subwords.

Why is this brilliant? 1. **Out-of-Vocabulary (OOV) words:** If the model has never seen "appled" before, it can still construct a meaningful vector because it has seen "app" and "ple" and "ed" in other words! 2. **Morphologically rich languages:** It performs wonderfully on languages with complex word formations (like German, Turkish, or Finnish).

Step-by-Step Code: Training FastText

```
from gensim.models import FastText

# 1. Use the same tokenized corpus from the Word2Vec example
# (Assume tokenized_corpus is already defined)

# 2. Train the FastText model
print("Training FastText model...")
ft_model = FastText(sentences=tokenized_corpus, vector_size=10, window=2, min_count=1)

# 3. Test on an Out-Of-Vocabulary (OOV) word!
# Notice that "doggy" is NOT in our training corpus.
oov_word = "doggy"
try:
    # Word2Vec would crash here with a KeyError. FastText handles it!
    vector = ft_model.wv[oov_word]
    print(f"\nSuccessfully generated vector for unknown word '{oov_word}':")
    print(vector)

    # Find similarities for the unknown word
    sims = ft_model.wv.most_similar(oov_word, topn=2)
    print(f"\nWords similar to '{oov_word}':", sims)
except KeyError:
    print(f"Error: {oov_word} not in vocabulary.")
```

Listing 3: Training a FastText model and handling OOV words

4 Summary: Which one should you choose?

- Use **Word2Vec** when you want a fast, reliable baseline and you have a domain-specific dataset.
- Use **GloVe** when you want to capture global corpus statistics and want to utilize standard pre-trained embeddings easily.
- Use **FastText** when your data has lots of typos, rare words, or you are working with languages that have complex prefixes and suffixes.

5 Your Assignment: Compare Embedding Models on Tech Support Data

Now it is your turn to apply what you have learned! Word embeddings are rarely the end goal; they are usually the *input* features for downstream NLP tasks.

Objective: Build and compare two Sentiment Classifiers (Resolved vs. Frustrated) using different word embeddings.

Instructions:

1. **Dataset (Customer Support):** Create a small python list containing 10 positive/resolved tech support statements (e.g., "The new patch fixed my router, thanks!", "Customer service was

incredibly helpful.”) and 10 negative/frustrated statements (e.g., ”My software keeps crashing after the update.”, ”I have been on hold for 45 minutes!”). Create a corresponding list of labels (1 for resolved, 0 for frustrated).

2. Embeddings (The Comparison):

- **Model A:** Load the pre-trained glove-twitter-25 model using `gensim.downloader`. (Notice we are using Twitter data here, which is much better suited for conversational support text than Wikipedia!).
 - **Model B:** Train a FastText model from scratch using your small dataset, or load `word2vec` `google-news-300` to see how formal news data handles casual complaints.
3. **Document Vectors:** Write a function that takes a sentence, fetches the vector for each word in the sentence using the chosen model, and calculates the *average* of those vectors to represent the whole sentence. Ensure you handle Out-Of-Vocabulary (OOV) words appropriately!
 4. **Classification:** Use `scikit-learn`’s `LogisticRegression`. Train two separate models: one using your GloVe-based sentence vectors and one using your FastText/Word2Vec sentence vectors.
 5. **Testing:** Pass a brand new, tricky support tweet (e.g., ”The wifi is behaving weirdly today, what is going on?”) through both pipelines. Print out the predictions. Which embedding model captured the nuance better?

Mentorship Note: Do not feel discouraged if the accuracy isn’t perfect on a tiny dataset. The goal here is to understand the *pipeline*: Text → Tokens → Embeddings → Classifier, and to see firsthand how different embedding sources (Twitter vs. News/Custom) affect real-world text!

”The limits of my language mean the limits of my world.” - Ludwig Wittgenstein 5

Lab Assignment 8: Contextual Embeddings (ELMo & BERT)

1 Introduction

Unlike static word embeddings (like Word2Vec or GloVe) where a word has the same vector representation regardless of its context, **Contextual Embeddings** generate dynamic representations. The vector for the word “bank” in “river bank” will be significantly different from the vector for “bank” in “bank account”. This lab explores two foundational architectures for generating contextual embeddings: ELMo (RNN-based) and BERT (Transformer-based).

2 ELMo: Embeddings from Language Models

ELMo relies on a bidirectional Long Short-Term Memory (BiLSTM) network trained on a massive text corpus to predict the next word. It considers the entire input sentence before assigning an embedding to each word.

2.1 When to use and why?

- **When to use:** When computational resources are strictly limited, or when maintaining legacy NLP systems built prior to the Transformer boom.
- **Why:** ELMo operates efficiently on standard GPUs and captures syntax and semantics effectively through character-level CNNs and BiLSTMs. However, in modern industry settings, it has largely been superseded by Transformer models.

2.2 Implementation (TensorFlow Hub)

The industry standard for using ELMo is via TensorFlow Hub. Below is the step-by-step implementation.

```
1 import tensorflow as tf
2 import tensorflow_hub as hub
3 import numpy as np
4
5 # 1. Load the ELMo model from TensorFlow Hub
6 # Industry Practice: Pin your model version (e.g., /3)
7 # to ensure reproducibility.
8 print("Loading ELMo model...")
9 elmo = hub.load("https://tfhub.dev/google/elmo/3")
10
11 # 2. Define real-life example sentences
12 # Notice how 'record' is used as a noun and a verb.
13 sentences = [
14     "I want to buy a new vinyl record.",
15     "Please record this meeting for the absent employees."
16 ]
17
18 # 3. Convert text to a TensorFlow constant tensor
19 text_tensor = tf.constant(sentences)
20
21 # 4. Extract Embeddings
22 # The 'default' signature processes a 1D tensor of strings.
23 embeddings_dict = elmo.signatures["default"](text_tensor)
```

```

25 # Extract the contextualized word embeddings
26 # Shape: (batch_size, max_length, 1024)
27 elmo_word_embeddings = embeddings_dict["elmo"]
28
29 print(f'Embedding shape: {elmo_word_embeddings.shape}')
30 # Access embedding for 'record' in sentence 1 vs sentence 2
31 # In a real pipeline, you would use these embeddings
32 # as input to a downstream Classifier.

```

Listing 1: Contextual Embeddings using ELMo

2.3 Important Changeable Parameters

- model_name: e.g., 'bert-base-uncased', 'bert-large-cased', or domain-specific like 'emilyalsentzer/Bio_ClinicalBERT'. Determines the size and pre-training data of the architecture.
- max_length: BERT supports a maximum of 512 tokens. You can set this lower (e.g., 128) to save memory if your texts are short (like tweets).
- output_hidden_states: If True, returns embeddings from all 12 layers (for base). *Why change it?:* Sometimes concatenating the last 4 layers gives better embeddings for specific token-level tasks like NER.

3 Lab Assignment

Scenario: You are an AI Engineer at a large E-commerce platform. The customer support team receives thousands of text reviews and tickets daily. They need an automated system to convert these reviews into rich contextual embeddings and classify them to route them to the correct department (e.g., Billing, Shipping, Product Defect).

Task:

1. Create a mock dataset of 10 customer reviews covering at least three categories (e.g., Billing, Shipping, Defect).
2. Use the Hugging Face transformers library to extract the [CLS] token embedding for each review using bert-base-uncased.
3. Write a simple function that calculates the *Cosine Similarity* between the embeddings of the reviews. Prove that reviews belonging to the same category (e.g., two shipping complaints) have a higher cosine similarity score than reviews from different categories.
4. Apply a clustering algorithm (e.g., K-Means) on the extracted BERT embeddings to automatically group the reviews into clusters.
5. Visualize the clustered embeddings in a 2D scatter plot using dimensionality reduction techniques like PCA or t-SNE to demonstrate how the different categories separate visually.

Prompt Engineering Integration: You are encouraged to use Large Language Models (ChatGPT, Gemini, Claude) to help you write the cosine similarity function or generate the mock dataset.

- **Requirement:** You must include a markdown cell in your notebook titled “AI Prompts Used” where you paste the exact prompts you fed to the AI and briefly explain how the AI’s output helped you solve the task.

Submission Guidelines:

- Complete the assignment in a Google Colab Notebook.
 - Name your file strictly as: USN_Name.ipynb (e.g., 1RV21CS001_JohnDoe.ipynb). -
- Ensure the notebook sharing settings are set to **“Anyone with the link can view”**.
- Submit the Google Colab URL to the respective assignment on Google Classroom (GCR).

Lab Assignment 9: Fine-Tuning BERT for Text Classification –

Transfer Learning with Transformers

Duration Prerequisites **Difficulty** 90–120 min Python, PyTorch basics
intermediate

Learning Objectives

- Understand the BERT architecture: tokenization, attention, and [CLS] token
- Load and preprocess data for BERT fine-tuning
- Fine-tune BertForSequenceClassification with Hugging Face Transformers
- Evaluate models using Accuracy, F1, and Confusion Matrix
- Run inference on custom text inputs

Prerequisites

Install before the lab:

```
pip install transformers datasets torch scikit-learn matplotlib seaborn
```

Note: GPU Recommended. In Google Colab → Runtime → Change runtime type → T4 GPU.

Contents

1 Install & Import Dependencies

2 Background: How BERT Works

3 Load and Explore the Dataset

4 Tokenization with BertTokenizer

5 Custom Dataset and DataLoaders

6 Load the Pre-Trained BERT Model

7 Optimizer and LR Scheduler**8 Training and Evaluation Loop****9 Plot Training Curves****10 Evaluate on the Test Set 7****11 Inference on Custom Sentences****12 Save and Reload the Model****13 Visualise Attention Weights (Bonus)****14 Summary****1 Install & Import Dependencies**

Run the cell below once to install all required packages, then import them. We also set random seeds for reproducibility and auto-detect the available device.

```
!pip install transformers datasets torch scikit-learn matplotlib seaborn --quiet

import torch
from transformers import BertTokenizer, BertForSequenceClassification
from transformers import get_linear_schedule_with_warmup
from datasets import load_dataset
from torch.utils.data import Dataset, DataLoader
from torch.optim import AdamW
from sklearn.metrics import accuracy_score, f1_score, classification_report

SEED = 42
torch.manual_seed(SEED)
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using: {DEVICE}")
```

2 Background: How BERT Works

BERT (Bidirectional Encoder Representations from Transformers) is a pre-trained language model introduced by Google in 2018. It learns deep contextual representations by training on massive text corpora with two self-supervised objectives: Masked Language Modelling (MLM) and Next Sentence Prediction (NSP).

Concept Explanation

WordPiece tokenization Words split into sub-word tokens. "unbelievably" → ["un", "##be", "liev", "##ably"]

[CLS] token Prepended to every input. Its final hidden state (768-dim) is used for classification.

[SEP] token Separates sentence pairs (e.g., for NLI tasks). Also appended at the end.

- Bidirectional attention** Every token attends to every other token simultaneously (unlike left-to-right GPT).
- Fine-tuning** Add a linear head on top of [CLS] and train on labelled data with a small LR.
- BERT-base config** 12 layers, 12 attention heads, 768 hidden size, 110M parameters.

BERT Architecture Diagram (Simplified)

Input: [CLS] I loved this film [SEP]

↓ *Token + Position + Segment Embeddings*

↓ *12 x Transformer Encoder Layers (Multi-Head Attention + FFN + LayerNorm)*

↓ *[CLS] hidden state (768-dimensional vector)*

↓ *Dropout → Linear (768 → 2) → Softmax*

Output: [0.08, 0.92] ⇒ **POSITIVE** (92% confident)

3 Load and Explore the Dataset

We use the **IMDb movie reviews dataset**: 50,000 reviews labelled as positive (1) or negative (0), split evenly into 25,000 train and 25,000 test. It is a classic binary sentiment classification benchmark.

```
raw_dataset = load_dataset("imdb")
print(raw_dataset)

# Inspect a sample
sample = raw_dataset["train"][0]
print("Label:", "POSITIVE" if sample["label"] == 1 else "NEGATIVE")
print("Text:", sample["text"][:300])
```

Dataset is perfectly balanced: 12,500 positive and 12,500 negative in both splits. –

Many reviews exceed 512 words - BERT will truncate them to 512 tokens. – We use a subset (2,000 train / 500 val / 500 test) for speed in this lab.

4 Tokenization with BertTokenizer

BERT uses WordPiece tokenization. Words not in the vocabulary are split into sub-word pieces prefixed with ##. The tokenizer also adds special tokens and pads/truncates sequences to a fixed length.

```

tokenizer = BertTokenizer . from_pretrained ("bert -base -uncased")
text = "The cinematography was absolutely unbelievable!"

# Step 1: tokenize into sub-words
tokens = tokenizer .tokenize(text)
# -> ["the", "cinema", "##tog", "##raphy", "was", "absolutely", "un", "##bel", ...]

# Step 2: full encoding with padding and truncation
encoding = tokenizer (
    text ,
    add_special_tokens =True , # adds [CLS] and [SEP]
    max_length =128 , # max length
    padding="max_length", # pad short sequences
    truncation =True , # truncate long sequences
    return_tensors ="pt" # return PyTorch tensors
)

# encoding["input_ids"] shape: (1, 128)
# encoding["attention_mask"] shape: (1, 128) -- 1 real , 0=pad

```

Term Description

input_ids Integer IDs for each token in the vocabulary
 attention_mask 1 for real tokens, 0 for [PAD] tokens (model ignores
 padding) token_type_ids 0 for sentence A, 1 for sentence B (for
 sentence-pair tasks) [PAD] token ID=0, used to fill sequences shorter
 than max_length

5 Custom Dataset and DataLoaders

PyTorch requires a custom Dataset class that inherits from `torch.utils.data.Dataset`. It must implement `__len__` and `__getitem__`. We then wrap it in a `DataLoader` for batching and shuffling.

```

class IMDbDataset (Dataset):
    def __init__(self, texts, labels, tokenizer, max_len):
        self.texts = texts
        self.labels = labels
        self.tokenizer = tokenizer
        self.max_len = max_len

    def __len__(self):
        return len(self.texts)

    def __getitem__(self, idx):
        encoding = self.tokenizer (
            str(self.texts[idx]),
            add_special_tokens=True,
            max_length=self.max_len,
            padding="max_length",
            truncation=True,
            return_tensors="pt"
        )
        return {
            "input_ids": encoding ["input_ids"].squeeze (0),
            "attention_mask": encoding ["attention_mask"].squeeze (0),
            "labels": torch.tensor(int(self.labels[idx]), dtype=torch.long)
        }

# Wrap in DataLoaders (Assuming train_dataset, val_dataset, test_dataset are instantiated)
train_loader = DataLoader (train_dataset, batch_size=16, shuffle=True, num_workers=2) val_loader = DataLoader
(val_dataset, batch_size=16, shuffle=False, num_workers=2) test_loader = DataLoader (test_dataset, batch_size=16,
shuffle=False, num_workers=2)

```

Hyperparameters Reference

Parameter Value Rationale

MAX_LEN 256 Balances coverage vs. memory. Use 512 for best accuracy.

BATCH_SIZE 16 Fits most 8GB GPUs. Reduce to 8 if OOM.

EPOCHS 3 BERT converges fast; 2-4 epochs is typical.

LR 2e-5 Recommended in the original BERT paper (1e-5 to 5e-5).

WARMUP_RATIO 0.1 10% of steps for warmup to stabilise early training.

6 Load the Pre-Trained BERT Model

BertForSequenceClassification is a wrapper that adds a linear classification head on top of the BERT encoder. The head is initialised randomly; the encoder weights come from pre-training on BookCorpus + Wikipedia.

```

model = BertForSequenceClassification . from_pretrained (
    "bert-base-uncased",
    num_labels=2, # binary classification
    output_attentions=False,
    output_hidden_states=False
)

model = model.to(DEVICE)

# Count parameters
total = sum(p.numel() for p in model.parameters())
trainable = sum(p.numel() for p in model.parameters() if p.requires_grad) print(f"Total: {total:,} | Trainable: {trainable:,}")
# -> Total: 109,483,778 | Trainable: 109,483,778

```

bert-base-uncased has 110M parameters. *bert-large-uncased* has 340M. For quick experiments, base is sufficient. All parameters are trainable by default. You can freeze lower layers to speed up training.

7 Optimizer and LR Scheduler

We use AdamW (Adam with decoupled weight decay) and a linear warmup + linear decay schedule. Bias and LayerNorm weights are excluded from weight decay (a common best practice).

```

no_decay = ["bias", "LayerNorm.weight"]
optimizer_grouped_parameters = [
    {
        "params": [p for n, p in model.named_parameters() if not any(nd in n for nd in no_decay)],
        "weight_decay": 0.01
    },
    {
        "params": [p for n, p in model.named_parameters() if any(nd in n for nd in no_decay)],
        "weight_decay": 0.0
    }
]

optimizer = AdamW(optimizer_grouped_parameters, lr=2e-5, eps=1e-8)

total_steps = len(train_loader) * EPOCHS
warmup_steps = int(total_steps * 0.1)

scheduler = get_linear_schedule_with_warmup(
    optimizer,
    num_warmup_steps=warmup_steps,
    num_training_steps=total_steps
)

```

Why Linear Warmup? At the start of fine-tuning, the classification head weights are random and can produce large gradients. A warmup phase gradually increases the learning rate from near-zero to the target LR, preventing destructive updates to the pre-trained encoder. After warmup, the LR decays linearly to zero by the end of training.

8 Training and Evaluation Loop

The training loop follows the standard PyTorch pattern. For each batch we: forward pass → compute loss → backward pass → clip gradients → optimizer step → scheduler step. After each epoch we evaluate on the validation set and save the best checkpoint.

```
for epoch in range(EPOCHS):
    model.train ()
    for batch in train_loader :
        input_ids = batch["input_ids"].to(DEVICE)
        attention_mask = batch["attention_mask"].to(DEVICE)
        labels = batch["labels"].to(DEVICE)

        optimizer . zero_grad ()
        outputs = model( input_ids=input_ids , attention_mask=attention_mask , labels=labels )
        loss = outputs.loss # cross -entropy loss
        loss.backward ()

        # Clip gradients to prevent exploding gradients
        torch.nn.utils. clip_grad_norm_ (model. parameters (), max_norm=1.0)

        optimizer .step ()
        scheduler .step ()

    # Evaluate on validation set after each epoch
    val_loss , val_acc , val_f1 = evaluate(model , val_loader , DEVICE)

# Evaluation Helper Function
def evaluate(model , data_loader , device):
    model.eval()
    all_preds , all_labels = [] , []

    with torch.no_grad ():
        for batch in data_loader :
            input_ids = batch["input_ids"].to(device)
            attention_mask = batch["attention_mask"].to(device)
            labels = batch["labels"].to(device)

            outputs = model( input_ids=input_ids , attention_mask=attention_mask , labels=labels )
            preds = torch.argmax(outputs.logits , dim =1)

            all_preds .extend(preds.cpu ().numpy ())
            all_labels .extend(labels.cpu ().numpy ())

    acc = accuracy_score (all_labels , all_preds )
    f1 = f1_score(all_labels , all_preds , average="macro")
    return acc , f1
```

- model.eval() disables dropout and batch norm updates during evaluation. –
- torch.no_grad() skips gradient computation, saving memory and time.
- torch.argmax(logits, dim=1) picks the class with the highest raw score. – Gradient clipping (max_norm=1.0) is essential for stable

BERT fine-tuning.

9 Plot Training Curves

After training, plot loss and accuracy curves for both train and validation sets. These help you diagnose overfitting, underfitting, and convergence speed.

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Loss curve
axes[0].plot(epochs, history["train_loss"], label="Train Loss")
axes[0].plot(epochs, history["val_loss"], label="Val Loss")
axes[0].set_title("Training vs Validation Loss")

# Accuracy curve
axes[1].plot(epochs, history["train_acc"], label="Train Accuracy")
axes[1].plot(epochs, history["val_acc"], label="Val Accuracy")
axes[1].set_title("Training vs Validation Accuracy")

plt.tight_layout()
plt.show()
```

10 Evaluate on the Test Set

Load the best checkpoint saved during training, then run a full evaluation on the held-out test set. Report per-class metrics and plot the confusion matrix.

```
# Load best checkpoint
model.load_state_dict(torch.load("best_bert_model.pt", map_location=DEVICE))

# Full test-set evaluation
test_loss, test_acc, test_f1, preds, true = evaluate(model, test_loader, DEVICE)
print(classification_report(true, preds, target_names=["Negative", "Positive"]))

# Confusion matrix
cm = confusion_matrix(true, preds)
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=["Negative", "Positive"],
            yticklabels=["Negative", "Positive"])
plt.title("Confusion Matrix")
plt.show()
```

Expected Results (2,000 train samples, 3 epochs)

Metric Expected Range Full Dataset (25k)

Accuracy 86–90% ~93%

F1 Macro 86–90% ~93%

Training Time ~5 min (GPU) ~45 min (GPU)

11 Inference on Custom Sentences

Once the model is trained, you can pass any text to it and get a prediction with a

confidence score.

```
def predict_sentiment (texts , model , tokenizer , device):
    model.eval()
    for text in texts:
        encoding = tokenizer (text , return_tensors ="pt", max_length =256 ,
                                padding="max_length", truncation =True)

        with torch.no_grad ():
            outputs = model( input_ids =encoding ["input_ids"].to(device),
                                attention_mask =encoding["attention_mask"].to(device))

            probs = torch.softmax(outputs.logits , dim =1).squeeze ()
            label = "POSITIVE" if torch.argmax(probs) == 1 else "NEGATIVE"
            print(f' {label} ( {probs.max().item():.2%}) - {text[:60]}')

predict_sentiment ([
    "An absolute masterpiece. The acting was phenomenal.",
    "Terrible plot , unbelievable characters. Waste of time.",
], model , tokenizer , DEVICE)
```

12 Save and Reload the Model

```
# Save (HuggingFace format saves config , weights , tokenizer)
model.save_pretrained ("./saved_bert_imdb")
tokenizer.save_pretrained ("./saved_bert_imdb")

# Reload
model2 = BertForSequenceClassification . from_pretrained ("./saved_bert_imdb")
tokenizer2 = BertTokenizer .
from_pretrained ("./saved_bert_imdb")
```

13 Visualize Attention Weights (Bonus)

BERT's attention weights show which tokens are "looking at" which other tokens. Pass `output_attentions=True` to the model and plot the attention matrix for a given layer and head using a heatmap.

```
attn_model = BertForSequenceClassification . from_pretrained (
    "./saved_bert_imdb", output_attentions =True).to(DEVICE)

with torch.no_grad ():
    outputs = attn_model (** encoding )

# outputs.attentions: tuple of 12 tensors , each (batch , heads , seq, seq)
attn = outputs.attentions [11][0 , 0].
cpu ().numpy () # layer 12, head 1

sns.heatmap(attn , xticklabels =tokens , yticklabels =tokens , cmap="viridis")
plt.title("Attention Weights Layer 12, Head 1")
plt.show ()
```

14 Summary

Step Topic Key Takeaway

0 Setup Install transformers, datasets, torch, sklearn 1 BERT Theory WordPiece, [CLS], bidirectional attention, 110M params 2 Data Exploration IMDb: 50k balanced reviews; many exceed 512 tokens 3 Tokenization BertTokenizer adds [CLS]/[SEP], pads/truncates to max_len 4 Data Pipeline Custom Dataset + DataLoader; batch_size=16, shuffle=True 5 Model Loading BertForSequenceClassification = encoder + linear head 6 Optimizer AdamW + linear warmup/decay; LR=2e-5; no decay on bias/LN 7 Training Loop Forward → loss.backward → clip_grad_norm → step 8 Training Curves Plot train/val loss and accuracy to diagnose overfitting 9 Test Evaluation classification_report + confusion matrix on held-out test 10 Inference predict_sentiment() returns label + softmax confidence 11 Save/Load save_pretrained / from_pretrained for HuggingFace format 12 Attention Maps output_attentions=True; visualise with seaborn heatmap

Assignment: Fine-Tune BERT on a New Multi-Class Problem

Dataset: ag_news (load_dataset("ag_news"))

120,000 news articles in 4 classes: World, Sports, Business, Sci/Tech

Deliverable: A single Jupyter Notebook with all tasks below completed.

1. **Task 1 – EDA:** Visualise the class distribution and word-length distribution for ag_news. Comment on any class imbalance and how it might affect training.
2. **Task 2 – Data Pipeline:** Create an AGNewsDataset class and DataLoaders for train/val/test splits. Use at least 3,000 training samples. Justify your choice of MAX_LEN.
3. **Task 3 – Model:** Load bert-base-uncased with num_labels=4. Print the total and trainable parameter count. Why is the number of trainable parameters the same as in the binary case?
4. **Task 4 – Training:** Fine-tune for at least 3 epochs using AdamW + linear warmup. Log training loss and validation accuracy at each epoch. Plot the learning-rate schedule.
5. **Task 5 – Evaluation:** Report per-class precision, recall, and F1 using classification_report. Plot the confusion matrix. Which class is hardest to classify?
6. **Task 6 – Inference:** Write at least 4 custom news headlines and predict their category with confidence scores. Include at least one ambiguous headline and discuss the result.
7. **Task 7 – Written Analysis (Markdown cell):** Answer:
 - (a) Which class was hardest, and why?
 - (b) How did test accuracy compare to the IMDb binary task, and what explains the

difference?

(c) What three changes would you make to improve performance?

Grading Rubric

Criterion Marks

Task 1 – EDA with visualisations and commentary 15

Task 2 – Correct Dataset class and DataLoaders 15

Task 3 – Model loads, num_labels=4, parameter count 10

Task 4 – Training converges; curves plotted 20

Task 5 – Evaluation metrics + confusion matrix 15

Task 6 – Inference with 4+ headlines and discussion 10

Task 7 – Written analysis (3 sub-questions) 15

Total 100

SUBMISSION: Upload your .ipynb file to the course portal. Include all cell outputs (run all cells before saving). Due date: As announced in class. Late submissions: -10 marks per day.